F056 – Métodos Computacionais em Física de Altas Energias

Lista 2: Estimação de Parâmetros

Prof. Marco Dias

2026

Instruções gerais

Você recebeu dados experimentais de medidas do tempo de vida de um elemento recém-descoberto. Os dados estão armazenados no arquivo `decay.txt` e consistem simplesmente em N medidas individuais do tempo de decaimento, em segundos.

Resolva os problemas abaixo em Python (`numpy`, `scipy` e `matplotlib`). O código, os gráficos gerados (em `.pdf`) e as respostas discursivas devem ser organizados em um repositório no GitHub, contendo:

- um script comentado, dividido por seções (uma por problema);
- os gráficos gerados, versionados no repositório;
- um arquivo `README.md` com as respostas discursivas e os valores estimados.

Problema 1

Desenhe o histograma da distribuição dos dados. Discuta qual seria a melhor escolha para o tamanho dos bins do histograma. Justifique sua resposta. Qual PDF teórica você escolheria para descrever os dados, e por quê?

Problema 2

Desenhe a PDF teórica escolhida para diferentes valores do parâmetro de interesse θ e discuta como a forma da distribuição muda. Calcule a probabilidade de medir um valor $t = 1\,\mathrm{s}$ ou menor, sob uma das hipóteses teóricas de sua escolha.

Problema 3

Dados os dados observados, você precisa escolher uma PDF teórica correta que os descreva bem – não esqueça de usar seu conhecimento de física! Depois de definida a PDF, use-a para derivar a função de Verossimilhança (*Likelihood*) para todos os valores medidos de t. Você consegue encontrar o máximo? Qual problema em potencial você poderia encontrar com essa abordagem?

Apenas para praticar, suponha que há uma única medida $t = 1\,\mathrm{s}$ no experimento. Desenhe a função de verossimilhança para essa medida única. Onde está o máximo da função de verossimilhança?

Problema 4

A partir da função de verossimilhança, defina uma função de Log-Verossimilhança (*Log-Likelihood*). Desenhe a função $-2\ln L$ para todos os valores medidos de t do arquivo fornecido. Usando $-2\ln L$, calcule o valor esperado para o tempo de vida médio e a incerteza correspondente, $\hat{\tau} \pm \sigma_{\hat{\tau}}$. Você acabou de realizar um ajuste de máxima verossimilhança *binned* (binado) ou *unbinned* (não binado)?

Problema 5*

Realize o ajuste de máxima verossimilhança **numericamente**: encontre o máximo da curva de log-verossimilhança e determine as incertezas a partir da própria curva $-2\ln L$.

Problema 6 – Distribuições

Desenhe a distribuição Gaussiana para três conjuntos diferentes de valores dos parâmetros (μ, σ) e explique como μ e σ afetam sua forma.

Se uma partícula recém-descoberta X é descrita por uma distribuição Gaussiana Gauss($\mu = 200\,\text{GeV}, \sigma = 2\,\text{GeV}$), calcule as seguintes probabilidades:

- de produzir a partícula X com massa de 205 GeV ou mais;
- de produzir independentemente duas partículas X com massas acima de 203 GeV cada.

Problema 7 – Distribuições (Teorema Central do Limite)

Vamos verificar que o Teorema Central do Limite funciona mesmo com a PDF exponencial $E(x;\lambda) = \lambda e^{-\lambda x}$. Gere um número aleatório x definido como $x = \frac{1}{3}(x_1 + x_2 + x_3)$, onde cada x_i é sorteado de $E(x;\lambda_i)$. Verifique a distribuição de x após gerar um grande número de eventos. Ela se parece com uma Gaussiana?

Agora refaça o experimento de Monte Carlo definindo $x = \frac{1}{N}\sum_{i=1}^{N} x_i$ e encontre o valor de N para o qual a distribuição resultante passa a se parecer com uma Gaussiana.

Suba seu programa na sua área do github em uma pasta chamada `estatistica_intro` e coloque no `README.md` nessa pasta respostas às perguntas feitas.

Solução

O tempo de decaimento de uma partícula/elemento instável é bem descrito por uma distribuição exponencial, consequência direta da taxa de decaimento constante por unidade de tempo (processo sem memória):

$$f(t;\tau) = \frac{1}{\tau}\, e^{-t/\tau}, \qquad \tau > 0,$$

onde τ é o tempo de vida médio, o parâmetro de interesse a ser estimado.

A função de verossimilhança para N medidas independentes $t_1, \ldots, t_N$ é

$$L(\tau) = \prod_{i=1}^{N} \frac{1}{\tau} e^{-t_i/\tau} = \frac{1}{\tau^N} \exp\left(-\frac{1}{\tau}\sum_{i=1}^{N} t_i\right),$$

e a log-verossimilhança

$$\ln L(\tau) = -N \ln \tau - \frac{1}{\tau}\sum_{i=1}^{N} t_i.$$

Maximizando $\ln L(\tau)$ (isto é, resolvendo $\partial \ln L/\partial\tau = 0$) obtém-se o estimador analítico

$$\hat{\tau} = \frac{1}{N}\sum_{i=1}^{N} t_i,$$

ou seja, a média amostral dos tempos medidos – este é um ajuste de máxima verossimilhança **não binado** (*unbinned*), pois usa diretamente os valores individuais t_i, e não um histograma. O código abaixo implementa os Problemas 1 a 4; a discussão do Problema 5 é apresentada na sequência.

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.integrate as integrate
import random
from scipy.optimize import brentq
from scipy.optimize import curve_fit
from scipy.special import erfcinv
# Problem 1
with open('decay.txt', 'r') as f:
    lines = (line.strip() for line in f if line)
    t = [float(line) for line in lines]
figure, axis = plt.subplots(ncols=2,figsize=(12,6))
axis[0].hist(t, bins = 50)
axis[0].set(xlabel='t [s]', ylabel='N')
axis[0].set_title("Histogram")
counts_obs,bin_edges_obs = np.histogram(t, 50)
bin_centres_obs = (bin_edges_obs[:-1] + bin_edges_obs[1:])/2.
err = np.sqrt(counts_obs)
axis[1].set_title("Histogram with error bars")
axis[1].errorbar(bin_centres_obs, counts_obs, yerr=err, fmt='o')
axis[1].set(xlabel='t [s]', ylabel='N')
plt.savefig("E2_Problem_1.pdf")
plt.clf()
# Problem 2
def decay_pdf(t,tau):
    return (1./tau)*np.exp(-1.0*t/tau)
x = np.linspace(0., 10., 1000)
f1 = decay_pdf(x, 1.0)
f2 = decay_pdf(x, 2.0)
f3 = decay_pdf(x, 0.5)
plt.plot(x, f1, label="$\tau$ = 1.0")
plt.plot(x, f2, label="$\tau$ = 2.0")
```

```
plt.plot(x, f3, label="$\tau$ = 0.5")
plt.xlabel('t [s]')
plt.ylabel('N')
plt.legend(loc="upper right")
plt.savefig("E2_Problem_2.pdf")
plt.clf()
#print(100*(integrate.quad(lambda x: decay_pdf(x, 2.0), 0.0, 1.0))[0])
# Problem 3
def Likelihood(tau, N, sum_t):
    return pow((1/tau),N)*np.exp(-sum_t/tau)
x = np.linspace(0.5, 1.5, 1000)
l = Likelihood(x, 1, 1.0)
plt.plot(x, l)
plt.xlabel('$\tau$ [s]')
plt.ylabel('L($\tau$)')
plt.savefig("E2_Problem_3.pdf")
plt.clf()
# Problem 4
def lnL(tau, N, sum_t):
    return -2.0*N*np.log(tau) - 2.0*sum_t/tau
x = np.linspace(1.0, 2.5, 1000)
logLikelihood = lnL(x, len(t), sum(t))
plt.plot(x, logLikelihood)
plt.xlabel('$\tau$ [s]')
plt.ylabel('2lnL($\tau$)')
plt.savefig("E2_Problem_4.pdf")
plt.clf()
index = np.argmax(np.array(logLikelihood))
tau_hat = x[index]
print("Maximum likeliheood estimator for mean lifetime is " + str(round(tau_hat
    ,4)))
def moved_lnL(tau):
    return lnL(tau, len(t), sum(t)) - logLikelihood[index] + 1.0
sigma_down = tau_hat - brentq(moved_lnL, 1.0, x[index])
sigma_up = brentq(moved_lnL, x[index], 1.5) - tau_hat
print("Sigma_up uncertainty = " + str(round(sigma_up,4)))
print("Sigma_dn uncertainty = " + str(round(sigma_down,4)))
```

Listing 1: Solução – Problemas 1 a 4

Comentários sobre cada problema:

- **Problema 1**: com poucos bins o histograma esconde detalhes da forma exponencial; com bins demais, a flutuação estatística (Poisson) domina e o histograma fica ruidoso. Uma escolha razoável é usar $N_{\text{bins}} \sim \sqrt{N}$ (regra usada no código, com 50 bins) ou a regra de Freedman–Diaconis, de modo que cada bin tenha um número razoável de eventos ($\gtrsim 10$) para que $\sigma_i = \sqrt{y_i}$ seja uma boa aproximação Gaussiana da incerteza de Poisson.

- **Problema 2**: a exponencial fica mais "achatada"(decai mais lentamente) quanto maior o τ. A probabilidade de medir $t \leq 1$ s sob a hipótese $\tau = 2.0$ é

$$P(t \leq 1) = \int_0^1 f(t; \tau = 2.0)\, dt = 1 - e^{-1/2} \approx 0.393,$$

  calculada numericamente por `scipy.integrate.quad` na linha comentada do código.

- **Problema 3**: o produto de N termos exponenciais pode facilmente estourar os limites de precisão numérica (*underflow*) para N grande – por isso, na prática, sempre se trabalha com $\ln L$ em vez de L. Para uma única medida $t = 1$ s, $L(\tau) = \tau^{-1} e^{-1/\tau}$ tem seu máximo exatamente em $\tau = 1$ s, coerente com $\hat{\tau} = \frac{1}{N}\sum t_i$ para $N = 1$.

- **Problema 4**: o ajuste realizado é **não binado**, pois $-2\ln L(\tau)$ é construído a partir da soma sobre todos os valores individuais t_i (`sum(t)`), e não a partir de um histograma. O estimador $\hat{\tau}$ é o valor de τ que minimiza $-2\ln L(\tau)$; a incerteza é obtida encontrando os pontos onde $-2\ln L(\tau) - (-2\ln L(\hat{\tau})) = 1$, correspondendo a um intervalo de 1σ.

Problema 5* – resolução numérica

Este problema pede exatamente o mesmo procedimento numérico já implementado no código do Problema 4: `np.argmax` sobre uma grade de valores de τ para encontrar $\hat{\tau}$, e `scipy.optimize.brentq` para localizar os pontos em que $-2\ln L(\tau)$ cruza $-2\ln L(\hat{\tau})+1$, o que define $\sigma_{\hat{\tau}}$ (intervalo de 1σ). Não há código adicional a apresentar aqui além do já mostrado acima – o objetivo do problema é que o(a) aluno(a) reconheça que essa abordagem numérica generaliza-se para casos em que não existe solução analítica fechada, como no modelo Breit-Wigner + fundo quadrático discutido em aula.

Problemas 6 e 7 – Distribuições

Para o Problema 6, a Gaussiana $\text{Gauss}(x;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$ é plotada para três pares (μ,σ): alterar μ apenas translada a curva ao longo do eixo x, sem mudar sua forma; alterar σ alarga (ou estreita) a curva, mantendo a área total sempre igual a 1. As probabilidades pedidas são calculadas integrando a cauda da Gaussiana com `scipy.integrate.quad`: a probabilidade de $m \geq 205$ GeV corresponde a 2.5σ acima da média ($\approx 0.62\%$), e a probabilidade de produzir *duas* partículas independentes com $m \geq 203$ GeV (1.5σ acima da média, $\approx 6.68\%$ cada) é o quadrado dessa probabilidade individual ($\approx 0.45\%$), pela regra do produto para eventos independentes.

Para o Problema 7, cada termo x_i da soma é sorteado de uma exponencial com uma escala diferente (`scale = j + 1.0`, ou seja, $\lambda_i = 1/(j+1)$): trata-se, portanto, de uma soma de variáveis aleatórias independentes, porém *não identicamente distribuídas*. Isso é proposital: o Teorema Central do Limite, em sua forma mais geral (Lindeberg–Feller), não exige que as x_i sejam i.i.d., apenas que sejam independentes e que nenhuma delas domine a variância total da soma. Para $N = 3$ o histograma de $x = \frac{1}{N}\sum_i x_i$ ainda é visivelmente assimétrico, refletindo a forma da exponencial; à medida que N cresce (30, 60, 100), a distribuição se aproxima cada vez mais de uma Gaussiana, ilustrando a convergência prevista pelo TCL mesmo quando as distribuições individuais não são iguais entre si.

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.integrate as integrate
import random
def Gaussian(x, mu, sigma):
    return (1.0/(sigma*np.sqrt(2*np.pi)))*np.exp(-(x-mu)**2/(2*sigma**2))
# Problem 1
x = np.linspace(-5,5,1000)
f1 = Gaussian(x,0.0,1.0)
f2 = Gaussian(x,1.0,1.0)
f3 = Gaussian(x,0.0,2.0)
plt.plot(x, f1, label = "mu = 0, sigma = 1")
plt.plot(x, f2, label = "mu = 1, sigma = 1")
plt.plot(x, f3, label = "mu = 0, sigma = 2")
plt.legend(loc="upper left")
plt.savefig('E1_Problem_1.pdf')
plt.clf()
print("Probability to produce the new particle with a mass of 205 GeV or more is
     " + str(round(100*(integrate.quad(lambda x: Gaussian(x,200.0, 2.0), 205.0,
    +np.inf))[0],2)) + "%.")
#print("Probability to produce the new particle with a mass with mass between
    199 and 201 GeV " + str(100*(integrate.quad(lambda x: Gaussian(x,200.0, 2.0)
    , 199.0, 201.0))[0]) + "%.")
```

```
print("Probability to produce the two particles with masses above 203 GeV is " +
     str(round(100*(integrate.quad(lambda x: Gaussian(x,200.0, 2.0), 203.0, +np.
    inf))[0]**2,2)) + "%.")
# Problem 2
x_histo_3 = []
x_histo_30 = []
x_histo_60 = []
x_histo_100 = []
for i in range(10000):
    x = 0.0
    N = 3
    for j in range(N):
        x = x + np.random.exponential(scale= j + 1.0, size=None)
    x_histo_3.append(x/N)
    x = 0.0
    N = 30
    for j in range(N):
        x = x + np.random.exponential(scale= j + 1.0, size=None)
    x_histo_30.append(x/N)
    x = 0.0
    N = 60
    for j in range(N):
        x = x + np.random.exponential(scale= j + 1.0, size=None)
    x_histo_60.append(x/N)
    x = 0.0
    N = 100
    for j in range(N):
        x = x + np.random.exponential(scale= j + 1.0, size=None)
    x_histo_100.append(x/N)
# Initialise the subplot function using number of rows and columns
figure, axis = plt.subplots(2, 2, constrained_layout=True)
axis[0, 0].hist(x_histo_3, bins=20)
axis[0, 0].set_title("N = 3")
axis[0, 0].set_xlabel("x")
axis[0, 1].hist(x_histo_30, bins=20)
axis[0, 1].set_title("N = 30")
axis[0, 1].set_xlabel("x")
axis[1, 0].hist(x_histo_60, bins=20)
axis[1, 0].set_title("N = 60")
axis[1, 0].set_xlabel("x")
axis[1, 1].hist(x_histo_100, bins=20)
axis[1, 1].set_title("N = 100")
axis[1, 1].set_xlabel("x")
plt.savefig('E1_Problem_2.pdf')
plt.clf()
```

Listing 2: Solução – Problemas 6 e 7 (Gaussiana e Teorema Central do Limite)

Note que, como esperado, os valores numéricos das probabilidades reproduzem exatamente o que foi obtido analiticamente acima: $\approx 0.62\%$ para $P(m \geq 205\,\text{GeV})$ e $\approx 0.45\%$ para as duas partículas com $m \geq 203\,\text{GeV}$ cada.